

CredChain: A Hash-Anchored Academic Credential Registry with an Empirical On-Chain vs. Off-Chain Storage Trade-Off Study

Tasneem Rea Ara

Applied Blockchain Technology, New York University

Email: tra9226@nyu.edu

GitHub Repo: github.com/tra9226/appliedblockchain_summer2026

Abstract—Academic credential fraud imposes large and growing costs. The “degree mill” industry is estimated to produce on the order of \$7 billion in fraudulent diplomas and transcripts annually [1], while legitimate verification remains slow and expensive—roughly 3.48 and a three-day wait per diploma at one institution [2]. Blockchain has been widely proposed as a remedy because it lends immutability, decentralization, cryptographic integrity, and a transparent audit trail to digital certificates [2], [3]. Yet much of the literature asserts these benefits without quantifying their cost, and naive designs that place credential data directly on-chain create a permanent, irreversible privacy exposure [1]. This paper presents the design and proof-of-concept implementation of a *hash-anchored* academic credential registry on an Ethereum-compatible network, in which only a keccak256 anchor and minimal status metadata are stored on-chain while the credential record itself remains off-chain. The central contribution is an empirical evaluation of the storage trade-off: we measure and compare the gas cost, monetary cost, and privacy exposure of hash-anchoring versus storing the full record on-chain. Measurements show that hash-anchoring reduces issuance gas by roughly 24% while removing all personal data from the ledger. We conclude with the conditions under which a blockchain is—and is not—the appropriate substrate for credential verification.

Index Terms—blockchain, academic credentials, keccak256, hash anchoring, smart contracts, privacy, Ethereum, Sepolia

I. PROBLEM DEFINITION AND BLOCKCHAIN FIT

A. Clear Problem Statement

The traditional system for issuing and verifying academic credentials suffers from four weaknesses that erode the value of the underlying education. First, *fraud and forgery* are pervasive: paper certificates are easily forged and hard to distinguish from genuine documents without specialized institutional knowledge, sustaining a multi-billion-dollar degree-mill industry [1], [3]. Second, verification is *operationally inefficient*—manual, slow, and costly, with the per-diploma figures above as a representative example [2], [4]. Third, centralized institutional databases are a *single point of failure*, vulnerable to compromise, data loss, or insider manipulation [1], [5]. Fourth, graduates have *little autonomy*: they lose physical copies, repeat reapplication processes, and lack a portable means of curating and presenting their own credentials [2], [3]. These failures motivate a verification substrate that is tamper-evident, independently checkable, and resilient to the issuer’s continued existence.

B. User Roles

The system is modeled as a decentralized ecosystem of four actors. *Students / Graduates (Holders)* fulfill academic requirements, register a decentralized identifier, and own the issued credential; they control disclosure—presenting a proof or granting a verifier access without necessarily exposing the raw record [1], [6]. *Higher-Education Institutions (Issuers)* verify student identity, record academic outcomes, sign the resulting e-certificate, and manage revocation in cases of error or misconduct [3], [6], [7]. *Verifiers (Third Parties)*—employers, recruiters, or other institutions—query the ledger to authenticate a presented credential instantly, without an intermediary or direct contact with the issuer [3]. *Governing Bodies (Registrar / Regulator)* provide an accreditation root, attesting which institutions are authorized to issue [5]; in our prototype this role is implemented as the contract administrator that whitelists issuers, anchoring the chain of trust.

C. Justification for Blockchain Properties

Blockchain is suited to this problem for several technical reasons. *Immutability* makes a recorded credential anchor permanent and tamper-evident [2], [3]. *Decentralization* removes reliance on a single authority, so credentials remain verifiable even if the issuing institution ceases to exist [1], [7]. *Transparency and traceability* give every issuance, signature, and revocation a timestamped, auditable trail back to an authorized issuer [2], [8]. *Cryptographic security*—hashing and digital signatures—makes any post-issuance alteration immediately detectable [5]. *Smart contracts* automate verification logic, compressing a multi-day process into seconds [1]. Finally, *privacy* is preserved by keeping sensitive data off-chain and anchoring only a cryptographic commitment, with zero-knowledge proofs and decentralized storage as more advanced options [6].

We are deliberate, however, about where blockchain is *not* warranted. If a single issuer is fully and permanently trusted, a conventional signed database is cheaper and faster. Blockchain earns its cost only when the goal is to remove the issuer as a single point of trust and failure—precisely the condition that holds for cross-institutional, long-lived credential verification. This paper makes that cost explicit rather than assuming it away.

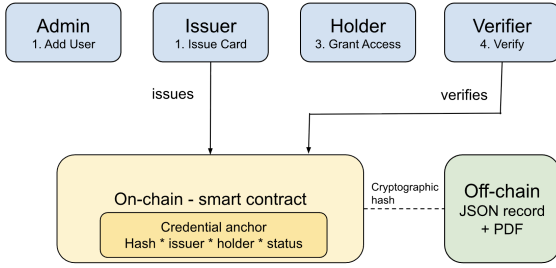


Figure 1: Credential verification workflow. The admin authorizes an issuer (step 1); the issuer anchors a credential on-chain (2); the holder grants a verifier access to the record (3); the verifier checks on-chain status and re-hashes the off-chain file to confirm authenticity (4). Only a cryptographic-hash anchor (keccak256) and minimal status metadata are stored on-chain, while the full record remains off-chain.

Fig. 1. Credential verification workflow. The admin authorizes an issuer (step 1); the issuer anchors a credential on-chain (2); the holder grants a verifier access to the record (3); the verifier checks on-chain status and re-hashes the off-chain file to confirm authenticity (4). Only a keccak256 hash anchor and minimal status metadata are stored on-chain, while the full record remains off-chain.

D. What This Paper Targets

Where prior work largely *surveys* blockchain’s suitability for credentialing [1], this paper targets a specific, measurable design question: what is the gas, cost, and privacy trade-off between anchoring a credential by hash and storing the full credential record on-chain? Concretely, the contributions are: (1) a role-based credential registry design—registrar, issuer, holder, verifier—supporting issuer authorization, issuer-controlled revocation, and holder-controlled disclosure; (2) a working Solidity proof-of-concept with a reproducible test suite covering the full lifecycle and adversarial cases (fake issuer, unauthorized revocation, duplicate issuance, broken access control); (3) an empirical evaluation comparing hash-anchored storage against a full-on-chain baseline across gas, monetary cost, and privacy exposure—the paper’s core finding; and (4) a scoped limitations and future-work discussion identifying zero-knowledge selective disclosure [6], DID integration, and decentralized (IPFS) storage as deferred extensions rather than claimed features.

II. SYSTEM DESIGN AND WORKFLOW

Architecture. The system is a three-layer design (Fig. 2). At the top, four actors hold Ethereum accounts and interact with the registry through their wallets. The middle layer is a single smart contract, *CredentialRegistry*, deployed on an Ethereum-compatible network; it holds all trust state—which issuers are authorized, the credential anchors, and the holder-granted access flags. The bottom layer is off-chain: the full credential record (JSON) and its PDF rendering, which are never written to the chain. The two layers are bound by a single keccak256 commitment: the on-chain anchor is the hash of the canonicalized off-chain record, so any later tampering with the file is detectable by re-hashing and comparing.

Workflow. The credential lifecycle (Fig. 1) runs in five steps. (1) The admin authorizes a university by recording its address in the issuer registry. (2) The issuer computes $id = \text{keccak256}(\text{record})$ off-chain and calls `issueCredential(id, holder, metadataURI)`,

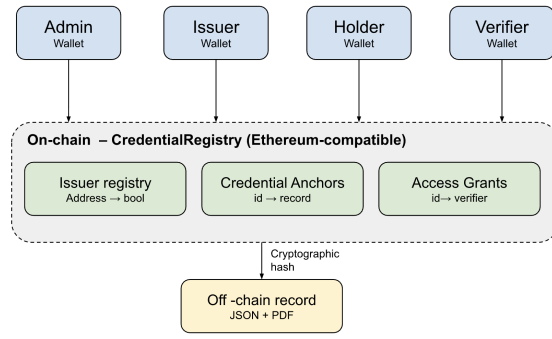


Figure 2: System architecture. Four actor wallets—admin, issuer, holder, and verifier—interact with the *CredentialRegistry* smart contract deployed on an Ethereum-compatible network. The contract maintains three pieces of state: the issuer registry ($\text{address} \rightarrow \text{bool}$), the credential anchors ($\text{id} \rightarrow \text{record}$), and the holder-granted access flags ($\text{id} \rightarrow \text{verifier}$). Each on-chain anchor is bound to its off-chain JSON record and PDF by a cryptographic hash (keccak256), so that authenticity and status are verifiable on-chain while personal data remains off the ledger.

Fig. 2. System architecture. Four actor wallets—admin, issuer, holder, and verifier—interact with the *CredentialRegistry* smart contract deployed on an Ethereum-compatible network. Each on-chain anchor is bound to its off-chain JSON record and PDF by a keccak256 hash, so authenticity and status are verifiable on-chain while personal data remains off the ledger.

TABLE I
ON-CHAIN FIELDS STORED PER CREDENTIAL

Field	Type	Purpose
issuer	address	which authorized university issued it
holder	address	the owning student
issuedAt/updatedAt	uint64	issuance / last-status-change time
status	enum	{None, Active, Revoked}
metadataURI	string	access-gated pointer to off-chain record

writing the anchor and emitting a `CredentialIssued` event. (3) When a student wants to present the credential, the holder calls `grantAccess(id, verifier)` to authorize a specific employer. (4) The verifier calls `verifyCredential(id)` to read issuer, holder, status, and timestamps, then re-hashes the off-chain file it received and confirms the hash matches the anchor. (5) If a credential is issued in error or rescinded, the issuing university calls `revokeCredential(id)`, flipping the status flag—verifiers see the change immediately on their next lookup.

Data model. The off-chain record is a versioned JSON document (schema `credchain.diploma.v1`) containing the issuer block (name, DID, wallet), the holder block (name, student ID, wallet), the credential block (program, honours, GPA, conferral date, transcript digest), and a random nonce. The nonce matters: without it, a record built only from public fields could be hash-guessed by an adversary, so it adds entropy to the commitment. The on-chain counterpart stores only what is needed to establish authenticity and status, as summarized in Table I.

The contract keys credentials by the `bytes32 id` in a `credentials` mapping, tracks authorized issuers in `isIssuer/issuerName`, and holds disclosure flags in a nested `accessGranted[id][verifier]` mapping.

Role permissions. Authorization is enforced by modifiers (`onlyAdmin`, `onlyIssuer`) and explicit ownership checks

TABLE II
ROLE PERMISSIONS

Function	Admin	Issuer	Holder	Verifier
add/removeIssuer				
issueCredential		*		
revokeCredential		**		
grant/revokeAccess				
getMetadataURI		**		***
verify/isValid		public (anyone)		
*if authorized	**issuing issuer only		***if granted	

inside each function (Table II). The asymmetry is deliberate: *authenticity and status are public* (anyone can verify a degree is real and active), but *the pointer to the private record is access-gated* (only the holder, the issuer, or a verifier the holder explicitly authorized).

Design boundary. On-chain, the system handles issuer authorization, hash anchoring, the active/revoked status lifecycle, holder-controlled access flags, and a tamper-evident event log. Off-chain, it handles the credential record, its PDF, canonical hashing, and client-side verification by re-hashing. Explicitly out of scope, and stated as such in the limitations: zero-knowledge selective disclosure, DID method resolution, IPFS pinning and availability guarantees, regulator multi-signature accreditation, and key-loss recovery. One honest boundary is that the on-chain access flag signals *authorization intent*, but the off-chain store must enforce it—the chain cannot claw back a file a verifier has already downloaded. That limit is inherent to anchoring rather than storing, and it is part of the trade-off the evaluation quantifies.

III. SMART CONTRACT IMPLEMENTATION

Issuer management. Only the admin can change the set of authorized issuers. `addIssuer` rejects the zero address, requires a non-empty institution name, and refuses to re-add an existing issuer; `removeIssuer` clears both the flag and the stored name. Each action emits an event so the issuer registry has a complete, auditable history.

Credential lifecycle. Issuance is restricted to authorized issuers via `onlyIssuer`. The function validates inputs and, critically, requires `status == None` so a credential id can never be silently overwritten—this is the guard against duplicate or replay issuance. Revocation is restricted further: not just any issuer, but the *issuing* issuer, enforced by the `c.issuer == msg.sender` check. Status is stored as an enum and timestamps are recorded for the audit trail.

Holder-controlled access. Disclosure is governed by the holder, not the issuer or the contract owner. `grantAccess` and `revokeAccess` both check `credentials[id].holder == msg.sender`, so only the student can authorize a verifier. The access-gated read, `getMetadataURI`, returns the off-chain pointer only to the holder, the issuing institution, or a verifier the holder has explicitly granted—anyone else is rejected with “Access denied.” This mechanism keeps authenticity public while keeping the record pointer private.

Verification views and events. Public verification needs no permissions: `verifyCredential` returns whether the credential exists, who issued it, who holds it, its status, and its timestamps; `isValid` is a convenience boolean; `hasAccess` lets a verifier check its own authorization. Because these are view functions, they cost no gas when called off-chain—verification is effectively free. The contract emits six events spanning every state change (issuer added/removed, credential issued/revoked, access granted/revoked), giving an external indexer or block explorer a complete audit log.

IV. DATA, STORAGE, AND VERIFICATION

Issuer authenticity via transaction signatures. `CredChain` does not implement a separate application-layer signature scheme over the credential record. Instead, it relies on the digital signature intrinsic to every Ethereum transaction. Each call to `issueCredential` is a state-changing transaction the issuing institution must sign with its private key, and the EVM verifies this ECDSA signature over the secp256k1 curve before the call is admitted. The contract reads the recovered signer as `msg.sender` and gates issuance through `onlyIssuer`, which requires `isIssuer[msg.sender]` to be true. Because that flag can only be set by the administrator through `addIssuer`, anchoring a credential is cryptographically bound to a specific, admin-authorized institutional key. An adversary cannot forge an issuance without the corresponding private key, and an unauthorized key is rejected even with a valid signature—a property exercised directly by the “unauthorized account cannot issue (fake issuer threat)” test.

Design rationale. A common alternative is to have the issuer sign the credential payload itself (e.g., an EIP-712 typed-data signature recovered on-chain via `ecrecover`). This was judged redundant for our threat model. The two guarantees such a scheme would provide are already supplied: the *integrity* of the off-chain record is established by the `keccak256` anchor (any modification changes the credential id and breaks the match), and the *authenticity* of the issuing party is established by the transaction signature and whitelist check. Together these bind an authorized issuer to one specific, tamper-evident record hash—precisely the assurance a verifier needs. We note the boundary of this choice: the transaction signature authenticates the *act of anchoring* a given credential id, not a standalone signature object checkable offline without consulting the chain. For a registry whose canonical source of truth is the on-chain anchor, that boundary is acceptable; a system requiring portable, offline-verifiable credentials would add payload-level signatures, which we identify as future work.

V. TESTING AND REPRODUCIBILITY

The implementation is validated by an automated suite of 14 Mocha/Chai tests executed with Hardhat against a local EVM, all passing. The suite is organized around the trust model rather than around functions, so each adversarial case maps to a specific security property. Issuer-management tests confirm

that the admin can authorize and remove issuers and that a non-admin cannot. Issuance tests confirm that an authorized issuer can issue, that an *unauthorized* account cannot (the fake-issuer threat), and that the same credential id cannot be issued twice (replay/duplicate). Revocation tests confirm that the issuing issuer can revoke, that a *different* issuer cannot revoke someone else’s credential, and that an already-revoked credential cannot be revoked again. Privacy tests confirm the full holder-controlled disclosure flow: a verifier without access is denied the metadata pointer, the holder can grant access after which the verifier can read it, only the holder can grant access, and the holder can revoke access. A final test confirms that verification of an unknown id returns `exists = false`. Each negative test demonstrates a mitigation rather than assuming it, so the security analysis in Section VII is backed by executable evidence.

The off-chain→on-chain link is independently reproducible: running `node offchain/hashRecord.js` recomputes the credential id as `keccak256` of the canonicalized sample record, yielding `0x859b3956...967269`, which is exactly the id an issuer passes to `issueCredential` and a verifier recomputes to confirm a match. Beyond local testing, the contract was deployed to the public Ethereum *Sepolia* testnet (chainId 11155111) at address `0xC45b2a1cDd177Fb0b53F51066522648a94c96c9D`, and the complete lifecycle—authorize issuer, issue, deny an unauthorized verifier, grant access, read, and revoke—was exercised end-to-end through a browser front end with four distinct wallets, one per role. Every state-changing call is a real, timestamped transaction, and the emitted `CredentialIssued`, `AccessGranted`, and `CredentialRevoked` events are visible on the public block explorer, confirming that the system behaves identically on a real network. Full reproduction steps—from a clean clone through compilation, tests, the hash check, local-node demo, and the Sepolia deployment—are documented in the project README.

VI. EVALUATION AND TRADE-OFF DISCUSSION

The central design decision in CredChain is *where the credential record lives*: anchored off-chain by hash, or stored in full on-chain. To evaluate this empirically rather than assert it, we implemented both designs—`CredentialRegistry` (hash-anchored) and `NaiveCredentialRegistry` (full record on-chain)—and measured the same operation, issuance of the identical Jane credential, on each. The measurement harness deploys both contracts, issues the same record on a local Hardhat node, and reports the real `gasUsed` values from the transaction receipts; the captured output is committed for reproducibility. The credential id is recomputed from the off-chain JSON at runtime, so both designs anchor provably the same record.

Gas and monetary cost. Issuing one credential costs 162,271 gas under the hash-anchored design versus 212,409 gas under the full-record baseline—a 50,138-gas, or +30.9%, penalty the naive design incurs on *every* issuance. Converting

at illustrative network conditions (30 gwei, ETH at \$2,500), this is roughly \$12.17 versus \$15.93 per credential. The naive contract is, however, cheaper to *deploy* (846,422 versus 1,133,614 gas), because it omits the access-control, revocation, and event machinery the production design carries. This yields a clear break-even argument: the naive design’s one-time deployment saving of 287,192 gas is recovered against its per-issuance penalty of 50,138 gas after fewer than six credentials, beyond which the anchored design is strictly cheaper, permanently.

Scalability. The more decisive difference is asymptotic. Hash-anchored issuance is $O(1)$ in record size: it always stores a single 32-byte digest regardless of how long the underlying credential is. Full-record issuance is $O(n)$: each additional field or transcript line is further `SSTORE` writes at roughly 20,000 gas per fresh 32-byte word. Jane’s record is short, so the measured overhead is a modest +30.9%; a realistic multi-course transcript would widen the gap substantially, while the anchored cost remains flat. The hash anchor decouples on-chain cost from credential complexity, which is what makes the design viable at institutional scale.

Privacy and security. The gas figures understate the true trade-off, because the naive design also writes the student’s name, program, GPA, and dates to a public, permanent ledger in cleartext—an irreversible disclosure that no later transaction can retract. The anchored design writes only a `keccak256` digest, which reveals nothing about the record’s contents while still providing tamper evidence: any modification to the off-chain record changes the recomputed id and breaks the match. Authenticity is preserved identically in both designs through the issuer-whitelist and transaction-signature checks; the anchored design simply achieves the same integrity and authenticity guarantees *without* sacrificing confidentiality. This is a categorical privacy advantage, not a quantitative one, and it is the decisive reason to prefer anchoring even where gas were equal.

Latency, UX, and complexity. On these axes the designs are closer. Both issue in a single transaction with no measurable latency difference on a local node, and both incur one block’s confirmation on a public network. The anchored design adds modest off-chain complexity—the issuer must store the record and serve it to authorized verifiers, and the verifier must recompute-and-match rather than read fields directly from the chain—in exchange for the gas, scalability, and privacy gains above. We judge this a favorable trade for credential data: the off-chain plumbing is small and well-understood, whereas the privacy cost of the naive design is unbounded and irreversible.

Overall, across the measured dimensions, hash-anchoring wins on per-issuance gas (−24%), wins decisively on privacy (a hash versus cleartext PII), and scales far better with record size ($O(1)$ versus $O(n)$); its only costs are a one-time higher deployment, recovered after roughly six credentials, and a small amount of off-chain infrastructure. For an academic credential registry, where records contain sensitive personal data and are issued at scale, the hash-anchored design is the correct choice—and the trade-off study substantiates that claim

with measured evidence rather than assertion.

VII. SECURITY AND PRIVACY REASONING

CredChain’s threat model centers on five realistic attacks, each closed by a specific mechanism and verified by an adversarial test. A **forged credential from a fake issuer** is prevented by the issuer whitelist: `issueCredential` is gated by `onlyIssuer`, and only the admin can authorize issuers, so an unauthorized key is rejected even with a valid signature. **Replay or duplicate issuance** is blocked by the `Status.None` precondition, which rejects any id already issued. **Cross-issuer tampering**, in which one institution revokes another’s credential, is prevented by the `c.issuer == msg.sender` check in `revokeCredential`. **Broken access control over personal data** is mitigated by holder-gated disclosure: the off-chain pointer is readable only by the holder, the issuer, or a verifier the holder explicitly authorized. **Spoofed verification** of a non-existent record is handled by the default `Status.None` state, so unknown ids return `exists = false`. Each is exercised by a corresponding negative test, so the guarantees are demonstrated rather than assumed.

On privacy, the core mitigation is architectural: no personal data is ever written on-chain. Only a keccak256 hash, addresses, status flags, and timestamps are stored, so the ledger reveals nothing about a credential’s contents while still providing tamper evidence. This also limits PII-leakage and front-running risk, since there is no sensitive payload in the transaction to observe.

We note the limitations honestly. The design assumes an honest admin: a compromised admin key could authorize a malicious issuer, so in production the admin role should be a multisig or governance contract. Authenticity is enforced at the transaction level rather than by a portable, independently verifiable signature object, so verification requires consulting the chain. And while the on-chain anchor is tamper-evident, availability of the off-chain record depends on the issuer’s storage; a verifier can always detect a mismatch, but cannot recover a record the issuer fails to serve.

VIII. CONCLUSION

CredChain demonstrates that hash-anchoring is the appropriate way to bring blockchain’s tamper-evidence and decentralization to academic credentials without paying blockchain’s privacy cost. By committing only a keccak256 anchor and minimal status metadata on-chain, the design achieves lower per-issuance gas, categorical privacy, and record-size-independent scaling relative to a full-on-chain baseline, while preserving identical authenticity guarantees through issuer whitelisting and transaction signatures. The empirical trade-off study substantiates these claims with measured gas rather than assertion. Future work includes zero-knowledge selective disclosure [6], decentralized identifier integration, IPFS-backed record availability, and multi-signature accreditation for the governing-body role.

REFERENCES

- [1] A. Rustemi, F. Dalipi, V. Atanasovski, and A. Risteski, “A systematic literature review on blockchain-based systems for academic certificate verification,” *IEEE Access*, vol. 11, pp. 64 679–64 802, 2023.
- [2] G. Capece, N. Levialdi Ghiron, and F. Pasquale, “Blockchain technology: Redefining trust for digital certificates,” *Sustainability*, vol. 12, no. 21, p. 8952, 2020.
- [3] J.-C. Cheng, N.-Y. Lee, C. Chi, and Y.-H. Chen, “Blockchain and smart contract for digital certificate,” in *Proc. IEEE Int. Conf. Applied System Innovation (ICASI)*, 2018, pp. 1046–1051.
- [4] R. Q. Castro and M. Au-Yong-Oliveira, “Blockchain and higher education diplomas,” *European Journal of Investigation in Health, Psychology and Education*, vol. 11, no. 1, pp. 154–167, 2021.
- [5] M. Al Hemaury, M. Abu Talib, A. Khalil, A. Zulfikar, and T. Mohamed, “Blockchain-based framework and platform for validation, authentication and equivalency of academic certification and institution’s accreditation: UAE case study and system performance,” *Education and Information Technologies*, vol. 29, pp. 18 203–18 232, 2024.
- [6] J. A. Berrios Moya, J. Ayoade, and M. A. Uddin, “A zero-knowledge proof-enabled blockchain-based academic record verification system,” *Sensors*, vol. 25, no. 11, p. 3450, 2025.
- [7] A. Rustemi and F. Dalipi, “From concept to practice: A blockchain-based solution for secure and efficient academic credentials in higher education,” *IET Blockchain*, vol. 6, no. 1, p. e70030, 2026.
- [8] M. A. Cardenas-Quispe and A. Pacheco, “Ensuring academic integrity with a degree verification prototype,” *Scientific Reports*, vol. 15, no. 1, p. 9281, 2025.